ЗАДАНИЕ

Построение и тестирование конечноэлементной вычислительной схемы с использованием векторных базисных функций на параллелепипедах.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. МАТЕМАТИЧЕСКАЯ ПОСТАНОВКА

Решение задачи в трехмерных декартовых координатах, удовлетворяющей расчетной области Ω, будем искать из следующего уравнения:

$$\mathrm{rot}\frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}} + \overrightarrow{\mathbf{A}} = \overrightarrow{\mathbf{J}} \tag{1.1}$$

Эквивалентная вариационная постановка для уравнения (1.1) имеет вид:

$$\int\limits_{\Omega} \frac{1}{\mu}\mathrm{rot}\,\overrightarrow{\mathbf{A}}\,\mathrm{rot}\,\overrightarrow{\Psi}\,d\Omega + \int\limits_{\Omega} \overrightarrow{\mathbf{A}}\,\overrightarrow{\Psi}\,d\Omega = \int\limits_{\Omega} \overrightarrow{\mathbf{J}}\,\overrightarrow{\Psi}\,d\Omega. \tag{1.2}$$

1.2. ПРИНЦИПЫ ПОСТРОЕНИЯ ЛОКАЛЬНЫХ ВЕКТОРОВ, МАТРИЦ ЖЕСТКОСТИ И МАСС В ВЕКТОРНОМ МКЭ

Решение будем искать, используя билинейные базисные вектор-функции, задаваемые линейными функциями на параллелепипиде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ следующего вида:

$$X_1(x) = \frac{x_{r+1} - x}{x_{r+1} - x_r}, \qquad X_2(x) = \frac{x - x_r}{x_{r+1} - x_r}, \tag{1.3}$$

$$Y_1(y) = \frac{y_{s+1} - y}{y_{s+1} - y_s}, \qquad Y_2(y) = \frac{y - y_s}{y_{s+1} - y_s}, \tag{1.4}$$

$$Z_1(z) = \frac{z_{p+1} - z}{z_{p+1} - z_p}, \qquad Z_2(z) = \frac{z - z_p}{z_{p+1} - z_p}. \tag{1.5}$$

Тогда базисная вектор-функция, ассоциированная с ребром $\{y = y_s, z = z_p\}$ имеет вид:

$$\overrightarrow{\psi}_1 = \begin{pmatrix} \frac{y_{s+1}-y}{h_y} \cdot \frac{z_{p+1}-z}{h_z} \\ 0 \\ 0 \end{pmatrix},$$

где $h_y = y_{s+1} - y_s$ $h_z = z_{p+1} - z_p$.

Полный список базисных вектор-функций на параллелепипиде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ можно записать в виде:

$$\overrightarrow{\psi}_1 = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_2 = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_3 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_4 = \begin{pmatrix} Y_2 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_5 = \begin{pmatrix} 0 \\ X_1 \cdot Z_1 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_6 = \begin{pmatrix} 0 \\ X_2 \cdot Z_1 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_7 = \begin{pmatrix} 0 \\ X_1 \cdot Z_2 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_8 = \begin{pmatrix} 0 \\ X_2 \cdot Z_2 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_9 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_{10} = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_{11} = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_{12} = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

Формулы для вычисления глобальных матриц жёсткости $\mathbf{G}$ и масс $\mathbf{M}$, определяющих глобальную матрицу $\mathbf{C} = \mathbf{G} + \mathbf{M}$ конечноэлеметной СЛАУ имеют вид:

$$G_{ij} = \int\limits_{\Omega} \frac{1}{\mu} \mathrm{rot}\overrightarrow{\Psi}_i \cdot \mathrm{rot}\overrightarrow{\Psi}_j d\Omega, \qquad M_{ij} = \int\limits_{\Omega} \gamma \overrightarrow{\Psi}_i \cdot \overrightarrow{\Psi}_j d\Omega. \tag{1.6}$$

Компоненты глобального вектора $\mathbf{b}$ конечноэлементной СЛАУ определяются соотношением:

$$b_i = \int\limits_{\Omega} \overrightarrow{\mathbf{F}} \cdot \overrightarrow{\Psi}_i d\Omega. \tag{1.7}$$

Локальная матрица жёсткости $\hat{\mathbf{G}}$ на векторном параллелепипеде при $\overline{\mu} = const$ принимает вид:

$$\hat{\mathbf{G}} = \frac{1}{\overline{\mu}} \begin{pmatrix} \frac{h_x h_y}{6h_z}\mathbf{G}_1 + \frac{h_x h_z}{6h_y}\mathbf{G}_2 & -\frac{h_z}{6}\mathbf{G}_2 & \frac{h_y}{6}\mathbf{G}_3 \\ -\frac{h_z}{6}\mathbf{G}_2 & \frac{h_x h_y}{6h_z}\mathbf{G}_1 + \frac{h_y h_z}{6h_x}\mathbf{G}_2 & -\frac{h_x}{6}\mathbf{G}_1 \\ \frac{h_y}{6}\mathbf{G}_3^{\mathrm{T}} & -\frac{h_x}{6}\mathbf{G}_1 & \frac{h_x h_z}{6h_y}\mathbf{G}_1 + \frac{h_y h_z}{6h_x}\mathbf{G}_2 \end{pmatrix},$$

где

$$\mathbf{G}_1 = \begin{pmatrix} 2 & 1 & -2 & -1 \\ 1 & 2 & -1 & -2 \\ -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \end{pmatrix}, \mathbf{G}_2 = \begin{pmatrix} 2 & -2 & 1 & -1 \\ -2 & 2 & -1 & 2 \\ 1 & -1 & 2 & -2 \\ -1 & 1 & -2 & 2 \end{pmatrix},$$

$$\mathbf{G}_3 = \begin{pmatrix} -2 & 2 & -1 & 1 \\ -1 & 1 & -2 & 2 \\ 2 & -2 & 1 & -1 \\ 1 & -1 & 2 & -2 \end{pmatrix}.$$

Матрицу $\hat{\mathbf{M}}$ можно представить в виде:

$$\mathbf{M} = \begin{pmatrix} \mathbf{D} & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{D} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{D} \end{pmatrix},$$

где $\mathbf{O}$ - матрица 4×4, состоящая из нулей подматрица, а $\mathbf{D}$ определяется следующим образом:

$$\mathbf{D} = \begin{pmatrix} 4 & 2 & 2 & 1 \\ 2 & 4 & 1 & 2 \\ 2 & 1 & 4 & 2 \\ 1 & 2 & 2 & 4 \end{pmatrix}.$$

Локальный вектор правой части определяется в виде $\hat{\mathbf{b}}_i = \hat{\mathbf{M}} \cdot \hat{\mathbf{f}}_i$, где

$$\hat{\mathbf{f}}_i = \overrightarrow{\mathbf{F}}(\hat{x}_c, \hat{y}_c, \hat{z}_c) \cdot \overrightarrow{\mathbf{v}}/l, \tag{1.8}$$

$\hat{x}_c, \hat{y}_c, \hat{z}_c$ - координаты центра ребра, $\overrightarrow{\mathbf{v}}$ - вектор, направленный вдоль ребра Γ и сонаправленный с базисной вектор-функцией $\overrightarrow{\psi}_i$, l - длина данного вектора.

2. ИССЛЕДОВАНИЯ

2.1. ПРЕДВАРИТЕЛЬНОЕ ОПИСАНИЕ

Проведем сначала тестирование разработанной программы по векторному МКЭ на работоспособность. Образец расчетной области изображен на рисунке 2.1. Это область $\Omega = [0.0, 2.0]_x \times [0.0, 2.0]_y \times [0.0, 2.0]_z$, она содержит 54 ребра, на всех границах будем задавать первые краевые условия. Середины наших элементов обозначены красным цветом, в этих точках мы будем рассматривать точность наших решений для оценки точности программы.

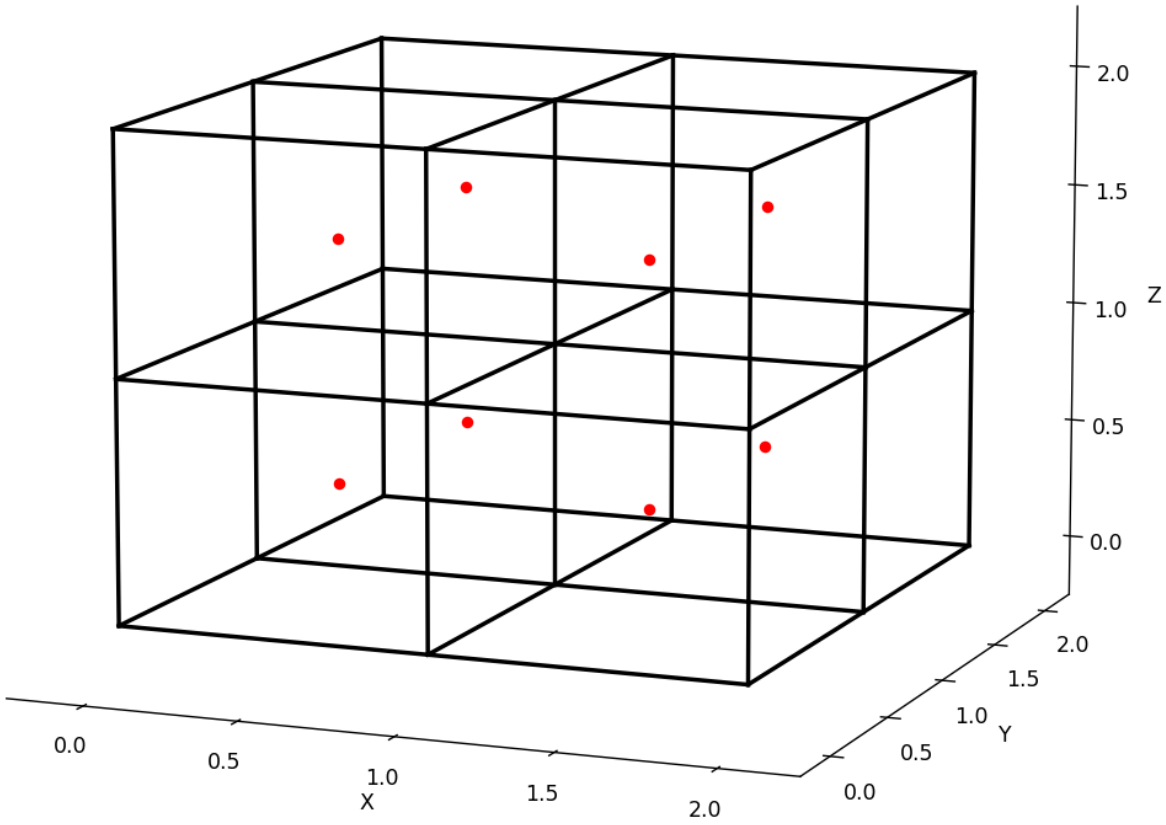

Рисунок 2.1 – Расчетная область

2.2. ТЕСТИРОВАНИЕ НА ПОРЯДОК АППРОКСИМАЦИИ

Тестирование будем проводить на дифференциальном уравнении (2.1) :

$$\mathrm{rot}\left(\frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}}\right) + \gamma\overrightarrow{\mathbf{A}} = \overrightarrow{\mathbf{F}} \tag{2.1}$$

Таблица 2.1 – Тестирование при $\overrightarrow{\mathbf{A}} = (1.0, 1.0, 1.0)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (1.0, 1.0, 1.0)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.2 – Тестирование при $\overrightarrow{\mathbf{A}} = (y, z, x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y, z, x)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.3 – Тестирование при $\overrightarrow{\mathbf{A}} = (1+y+x; 1+x+z; 1+x+y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (1+y+x; 1+x+z; 1+x+y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.00000000E+000; 2.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	3.00000000E+000; 2.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	4.00000000E+000; 3.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	2.00000000E+000; 3.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 4.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	4.00000000E+000; 4.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.4 – Тестирование при $\overrightarrow{\mathbf{A}} = (y - z; x - z; x - y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y - z; x - z; x - y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	0.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 0.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	-1.00000000E+000; -1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	-1.00000000E+000; 0.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	0.00000000E+000; -1.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.5 – Тестирование при $\overrightarrow{\mathbf{A}} = (y \cdot z; x \cdot z; x \cdot y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y \cdot z; x \cdot z; x \cdot y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	2.50000000E-001; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	7.50000000E-001; 2.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	7.50000000E-001; 7.50000000E-001; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	7.50000000E-001; 7.50000000E-001; 2.500000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	7.50000000E-001; 2.25000000E+000; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	2.25000000E+000; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	2.25000000E+000; 2.25000000E+000; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.6 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2; z^2; x^2)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y^2 - 2; z^2 - 2; x^2 - 2)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.11111111E-001
(0.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.00000000E+000; 1.00000000E+000
(1.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.00000000E+000; 1.111111111E-001
(0.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 5.000000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.111111111E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.111111111E-001; 1.111111111E-001
(0.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.111111111E-001; 1.00000000E+000
(1.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.111111111E-001; 1.111111111E-001

Таблица 2.7 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2 + z^2; x^2 + z^2; x^2 + y^2)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y^2 + z^2 - 4; x^2 + z^2 - 4; x^2 + y^2 - 4)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 2.00000000E-001; 2.00000000E-001
(0.5; 1.5; 0.5)	3.00000000E+000; 1.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.00000000E+000; 2.00000000E-001
(1.5; 1.5; 0.5)	3.00000000E+000; 3.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.111111111E-001
(0.5; 0.5; 1.5)	3.00000000E+000; 3.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 5.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.111111111E-001; 2.00000000E-001
(0.5; 1.5; 1.5)	5.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.111111111E-001; 2.00000000E-001; 2.00000000E-001
(1.5; 1.5; 1.5)	5.00000000E+000; 5.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.111111111E-001; 1.111111111E-001; 1.111111111E-001

Порядок аппроксимации равен 1.

2.3. ТЕСТИРОВАНИЕ НА ПОРЯДОК СХОДИМОСТИ

Таблица 2.8 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2z^2; x^2z^2; x^2y^2)^{\mathrm{T}}$,
$\overrightarrow{\mathbf{F}} = (y^2z^2 - 2\cdot(y^2+z^2); x^2z^2 - 2\cdot(x^2+z^2); x^2y^2 - 2\cdot(x^2+y^2))^{\mathrm{T}}$,
$\mu = 1$, $\gamma = 1$

Точка	Относительная погрешность	Порядок $\log_2\left(\frac{\Delta i-1}{\Delta_i}\right)$
$(^4/_3;\ ^4/_3;\ ^4/_3)$	2.6562500E-001 2.6562500E-001 2.6562500E-001	—
	6.3476562E-002 6.3476562E-002 6.3476562E-002	2.065
	1.5686035E-002 1.5686035E-002 1.5686035E-002	2.017
	3.9100647E-003 3.9100647E-003 3.9100647E-003	2.004

Можем увидеть, что порядок сходимости стремится к 2.

СПИСОК ЛИТЕРАТУРЫ

1. Ю.Г. Соловейчик, М.Э. Рояк, М.Г. Персова Метод конечных элементов для скалярных и векторных задач Учеб. пособие. — Новосибирск: Изд-во НГТУ, 2007 — 896 с.

2. А.Н. Тихонов, А.А. Самарский Уравнения математической физики: Учеб.пособие. / А.Н. Тихонов, А.А. Самарский — 6-е изд., — М: Изд-во МГУ, 1999 — 799 с.

ПРИЛОЖЕНИЕ. ТЕКСТ ПРОГРАММЫ.

Program.cs

```
#define TESTING
// RELEASE
// TESTING
#define NODRAWING
#define SOLVE2DIM
using Project;
using System.Globalization;
using Solver;
using Manager;
using System.Numerics;
using Processor;
using Solution;
using MathObjects;
using Functions;
using Grid;
using DataStructs;


CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;

string BordersInfo = Path.GetFullPath("../../../../Data/Input/Borders.dat");
string AnswerPath = Path.GetFullPath("../../../../Data/Output/");
string TimePath = Path.GetFullPath("../../../../Data/Input/Time.dat");

FEM3D myFEM3D_test = new();
myFEM3D_test.ConstructMesh(3, 3, 3);
myFEM3D_test.GenerateArrays();
myFEM3D_test.ConstructMatrixAndVector();
myFEM3D_test.SetSolver(new LOS());
myFEM3D_test.Solve();
myFEM3D_test.TestOutput(AnswerPath);
myFEM3D_test.WriteData(AnswerPath);

return 0;
```

LocalMatrix3D.cs

```
namespace MathObjects;

public class LocalMatrixG3D (double mu, double hx, double hy, double hz) : Matrix
{
    private readonly double _mu = mu;
    private readonly double _hx = hx;
    private readonly double _hy = hy;
    private readonly double _hz = hz;

    private readonly double[,] G1 = {{ 2.0D,  1.0D, -2.0D, -1.0D},
                                     { 1.0D,  2.0D, -1.0D, -2.0D},
                                     {-2.0D, -1.0D,  2.0D,  1.0D},
                                     {-1.0D, -2.0D,  1.0D,  2.0D}};

    private readonly double[,] G2 = {{ 2.0D, -2.0D,  1.0D, -1.0D},
                                     {-2.0D,  2.0D, -1.0D,  1.0D},
                                     { 1.0D, -1.0D,  2.0D, -2.0D},
                                     {-1.0D,  1.0D, -2.0D,  2.0D}};

    private readonly double[,] G3 = {{-2.0D,  2.0D, -1.0D,  1.0D},
                                     {-1.0D,  1.0D, -2.0D,  2.0D},
                                     { 2.0D, -2.0D,  1.0D, -1.0D},
                                     { 1.0D, -1.0D,  2.0D, -2.0D}};

    public override double this[int i, int j]
    {
        get
        {
            return (i / 4, j / 4)
            switch
            {
                (0, 0) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hx * _hz /
                ↪  (6.0D * _hy) * G2[i % 4, j % 4]) / _mu,
                (0, 1) or (1, 0) => -1.0D * (_hz / 6.0D) * G2[i % 4, j % 4] / _mu,
                (0, 2) => _hy / 6.0D * G3[i % 4, j % 4] / _mu,
                (1, 1) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hy * _hz /
                ↪  (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
                (1, 2) or (2, 1) => -1.0D * _hx / 6.0D * G1[i % 4, j % 4] / _mu,
```

```
                (2, 0) => _hy / 6.0D * G3[j % 4, i % 4] / _mu,
                (2, 2) => (_hx * _hz / (6.0D * _hy) * G1[i % 4, j % 4] + _hy * _hz /
                ↪  (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
                _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
                ↪  range"),
            };
        }
        set{}
    }
}

public class LocalMatrixM3D(double gamma, double hx, double hy, double hz) : Matrix
{
    private readonly double _gamma = gamma;
    private readonly double _hx = hx;
    private readonly double _hy = hy;
    private readonly double _hz = hz;

    private readonly double[,] D = {{4.0D, 2.0D, 2.0D, 1.0D},
                                    {2.0D, 4.0D, 1.0D, 2.0D},
                                    {2.0D, 1.0D, 4.0D, 2.0D},
                                    {1.0D, 2.0D, 2.0D, 4.0D}};

    public override double this[int i, int j]
    {
        get
        {
            return (i / 4, j / 4)
            switch
            {
                (0, 0) or (1, 1) or (2, 2) => _gamma * _hx * _hy * _hz / 36.0D * D[i % 4,
                ↪  j % 4],
                (0, 1) or (0, 2) or (1, 0) or (1, 2) or (2, 0) or (2, 1) => 0.0D,
                _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
                ↪  range"),
            };
        }
        set{}
    }
}
```

LocalVector3D.cs

```
using static Functions.Function;

namespace MathObjects;


public class LocalVector3D : Vector
{
    private readonly double x0;
    private readonly double x1;
    private readonly double y0;
    private readonly double y1;
    private readonly double z0;
    private readonly double z1;
    private readonly double xm;
    private readonly double ym;
    private readonly double zm;
    private readonly double t;

    private LocalMatrixM3D M;

    public override double this[int i]
    {
        get
        {
            double ScalarMult((double, double, double) a, (double, double, double) b)
            => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;

            (double, double, double) Divide ((double, double, double) v, double sc)
            => (v.Item1 / sc, v.Item2 / sc, v.Item3 / sc);

            double Length((double, double, double) v)
            => Math.Sqrt(Math.Pow(v.Item1, 2) + Math.Pow(v.Item2, 2) + Math.Pow(v.Item3,
            ↪  2));

            // Очень спорно.
            List<double> vect = [
                ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
                ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
```

```
                ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
                ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),

                ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),

                ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
            ];

            double ans = 0.0D;
            for (int j = 0; j < vect.Count; j++)
                ans += M[i, j] * vect[j];
            return ans;
        }
    }

    public LocalVector3D(double x0, double x1, double y0, double y1, double z0, double
    ↪  z1, double t)
    {
        this.x0 = x0;
        this.x1 = x1;
        this.y0 = y0;
        this.y1 = y1;
        this.z0 = z0;
        this.z1 = z1;
        this.t = t;
        xm = 0.5D * (x1 + x0);
        ym = 0.5D * (y1 + y0);
        zm = 0.5D * (z1 + z0);
        M = new(1.0, x1 - x0, y1 - y0, z1 - z0);
        Generate();
    }

    private void Generate()
    {
```

```
        double ScalarMult((double, double, double) a, (double, double, double) b)
        => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;

        List<double> vect = [
            ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
            ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
            ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
            ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),

            ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
            ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
            ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
            ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),

            ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
            ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
            ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
            ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
        ];

        double ans = 0.0D;
        for (int i = 0; i < vect.Count; i++)
        {
            for (int j = 0; j < vect.Count; j++)
                ans += M[i, j] * vect[j];
            ans = 0.0D;
        }
    }
}
```

FEM3D.cs

```
namespace Project;

using System.Collections.Immutable;
using System.Numerics;
using MathObjects;
using Solver;
using Grid;
using DataStructs;
```

```
using System.Diagnostics;
using Functions;
using System.ComponentModel.DataAnnotations;
using System.Timers;

public class FEM3D : FEM
{
    public ArrayOfRibs ribsArr;

    // Maybe private?
    public List<Layer> Layers;

    public FEM3D()
    {
        Layers = [];
        mesh = new Mesh3Dim
        {
            nodesX = [],
            nodesY = [],
            nodesZ = []
        };
    }

    public void ConstructMesh(int nx, int ny, int nz)
    {
        if (mesh == null) throw new ArgumentNullException("Mesh is null");

        double hx = 2.0D;
        double hy = 2.0D;
        double hz = 2.0D;

        mesh.nodesX = [];
        mesh.nodesY = [];
        mesh.nodesZ = [];

        for (int i = 0; i < nx; i++)
        {
            double stepx = hx / (nx - 1);
            mesh.nodesX.Add(i * stepx);
        }
```

```
49
50          for (int i = 0; i < ny; i++)
51          {
52              double stepy = hy / (ny - 1);
53              mesh.nodesY.Add(i * stepy);
54          }
55
56          for (int i = 0; i < nz; i++)
57          {
58              double stepz = hz / (nz - 1);
59              mesh.nodesZ.Add(i * stepz);
60          }
61
62          timeMesh = [1.0D];
63      }
64
65
66
67      public void GenerateArrays()
68      {
69          if (mesh is null) throw new ArgumentNullException("mesh is null!");
70          pointsArr = MeshGenerator.GenerateListOfPoints(mesh);
71          ribsArr = MeshGenerator.GenerateListOfRibs(mesh, pointsArr);
72          elemsArr = MeshGenerator.GenerateListOfElems(mesh, ribsArr);
73          bordersArr = MeshGenerator.GenerateListOfBorders(mesh);
74      }
75
76      public void AddField(Layer layer)
77      {
78          ArgumentNullException.ThrowIfNull(layer);
79          Layers.Add(layer);
80      }
81
82      public void CommitFields()
83      {
84          if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
85
86          foreach (var layer in Layers)
87          {
88              for (int i = 0; i < elemsArr.Length; i++)
```

```
89              {
90                  double minz = Math.Min(ribsArr[elemsArr[i][^1]].a.Z,
                    ↪  ribsArr[elemsArr[i][^1]].b.Z);
91                  double maxz = Math.Max(ribsArr[elemsArr[i][^1]].a.Z,
                    ↪  ribsArr[elemsArr[i][^1]].b.Z);
92                  if (layer.z0 <= minz && maxz <= layer.z1)
93                      elemsArr.sigmai[i] = layer.sigma;
94              }
95          }
96      }
97
98      public void ConstructMatrixAndVector()
99      {
100         if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
101         if (bordersArr is null) throw new ArgumentNullException("bordersArr is null");
102
103         var sparceMatrix = new GlobalMatrix(ribsArr.Count);
104         Generator.BuildPortait(ref sparceMatrix, ribsArr.Count, elemsArr);
105
106         var G = new GlobalMatrix(sparceMatrix);
107         Generator.FillMatrixG(ref G, ribsArr, elemsArr);
108
109         var M = new GlobalMatrix(sparceMatrix);
110         Generator.FillMatrixM(ref M, ribsArr, elemsArr);
111
112         Matrix = G + M;
113
114         var b = new GlobalVector(ribsArr.Count);
115         Generator.FillVector3D(ref b, ribsArr, elemsArr, 0.0);
116         Vector = b;
117
118         Generator.ConsiderBoundaryConditions(ref Matrix, ref Vector, ribsArr, bordersArr,
            ↪  0.0D);
119     }
120
121     public void Solve()
122     {
123         if (solver is null) throw new ArgumentNullException("Solver is null");
124         if (Matrix is null) throw new ArgumentNullException("Matrix is null");
125         if (Vector is null) throw new ArgumentNullException("Vector is null");
```

```
126             Solutions = new GlobalVector[timeMesh.Length];
127             Discrepancy = new GlobalVector[timeMesh.Length];
128             (Solutions[0], Discrepancy[0]) = solver.Solve(Matrix, Vector);
129         }
130 
131         public void TestOutput(string path)
132         {
133             using var sw = new StreamWriter(path + "/Answer3D/Answer_Test.txt");
134 
135             var absDiscX = 0.0D;
136             var absDiscY = 0.0D;
137             var absDiscZ = 0.0D;
138 
139             var absDivX = 0.0D;
140             var absDivY = 0.0D;
141             var absDivZ = 0.0D;
142 
143             var relDiscX = 0.0D;
144             var relDiscY = 0.0D;
145             var relDiscZ = 0.0D;
146 
147             var relDivX = 0.0D;
148             var relDivY = 0.0D;
149             var relDivZ = 0.0D;
150 
151             var squareDiffX = 0.0D;
152             var squareDiffY = 0.0D;
153             var squareDiffZ = 0.0D;
154 
155             int iter = 0;
156 
157             foreach (var elem in elemsArr)
158             {
159                 int[] elem_local = [elem[0], elem[3], elem[8], elem[11],
160                                     elem[1], elem[2], elem[9], elem[10],
161                                     elem[4], elem[5], elem[6], elem[7]];
162 
163                 var x = 0.5D * (ribsArr[elem_local[0]].a.X + ribsArr[elem_local[0]].b.X);
164                 var y = 0.5D * (ribsArr[elem_local[4]].a.Y + ribsArr[elem_local[4]].b.Y);
165                 var z = 0.5D * (ribsArr[elem_local[8]].a.Z + ribsArr[elem_local[8]].b.Z);
```

```
166
167             sw.WriteLine($"Points {x:E15} {y:E15} {z:E15}");
168
169             var eps = (x - ribsArr[elem_local[0]].a.X) / (ribsArr[elem_local[0]].b.X -
                ↪  ribsArr[elem_local[0]].a.X);
170             var nu =  (y - ribsArr[elem_local[4]].a.Y) / (ribsArr[elem_local[4]].b.Y -
                ↪  ribsArr[elem_local[4]].a.Y);
171             var khi = (z - ribsArr[elem_local[8]].a.Z) / (ribsArr[elem_local[8]].b.Z -
                ↪  ribsArr[elem_local[8]].a.Z);
172
173             double[] q = [Solutions[0][elem_local[0]], Solutions[0][elem_local[1]],
                ↪  Solutions[0][elem_local[2]], Solutions[0][elem_local[3]],
174                           Solutions[0][elem_local[4]], Solutions[0][elem_local[5]],
                               ↪  Solutions[0][elem_local[6]], Solutions[0][elem_local[7]],
175                           Solutions[0][elem_local[8]], Solutions[0][elem_local[9]],
                               ↪  Solutions[0][elem_local[10]], Solutions[0][elem_local[11]]];
176
177             var ans = BasisFunctions3DVec.GetValue(eps, nu, khi, q);
178             var theorValue = Function.A(x, y, z, 0.0D);
179
180             sw.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15} {ans.Item3:E15}");
181             sw.WriteLine($"Theor  {theorValue.Item1:E15} {theorValue.Item2:E15}
                ↪  {theorValue.Item3:E15}");
182
183             var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
184             var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
185             var currAbsDiscZ = Math.Abs(ans.Item3 - theorValue.Item3);
186
187             squareDiffX += currAbsDiscX * currAbsDiscX;
188             squareDiffY += currAbsDiscY * currAbsDiscY;
189             squareDiffZ += currAbsDiscZ * currAbsDiscZ;
190
191             var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
192             var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
193             var currRelDiscZ = currAbsDiscZ / Math.Abs(theorValue.Item3);
194
195             sw.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}
                ↪  {currAbsDiscZ:E15}");
196             sw.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}
                ↪  {currRelDiscZ:E15}\n");
```

```
197
198             absDiscX += currAbsDiscX;
199             absDiscY += currAbsDiscY;
200             absDiscZ += currAbsDiscZ;
201
202             absDivX += theorValue.Item1;
203             absDivY += theorValue.Item2;
204             absDivZ += theorValue.Item3;
205
206             relDiscX += currRelDiscX * currRelDiscX;
207             relDiscY += currRelDiscY * currRelDiscY;
208             relDiscZ += currRelDiscZ * currRelDiscZ;
209
210             relDivX += theorValue.Item1 * theorValue.Item1;
211             relDivY += theorValue.Item2 * theorValue.Item2;
212             relDivZ += theorValue.Item3 * theorValue.Item3;
213
214             iter++;
215         }
216         sw.WriteLine($"Avg disc: {absDiscX / iter:E15} {absDiscY / iter:E15} {absDiscZ /
            ↪ iter:E15}");
217         sw.WriteLine($"Rel disc: {Math.Sqrt(relDiscX / relDivX):E15} {Math.Sqrt(relDiscY
            ↪ / relDivY):E15} {Math.Sqrt(relDiscZ / relDivZ):E15}");
218         sw.WriteLine($"SKO: {Math.Sqrt(squareDiffX / elemsArr.Length):E15}
            ↪ {Math.Sqrt(squareDiffY / elemsArr.Length):E15} {Math.Sqrt(squareDiffZ /
            ↪ elemsArr.Length):E15}");
219         sw.Close();
220     }
221 }
```